Netflix Business Case: Data Exploration and Visualisation

Problem Statement

Netflix has more than 8,000 movies and TV shows. The business wants to know:

1. What type of content should Netflix produce?
2. How can Netflix grow in different countries?

I will clean the data, study it with numbers and charts, and then give simple insights and recommendations.

About the Data

The data has 8807 rows (one row is one title) and 12 columns: show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in (genre) and description.

In [6]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
df = pd.read_csv('netflix.csv')
df.head()
```

Out[6]:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	liste
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Document
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	Internat TV Show Drama Myst
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crim Sh Internat TV Show
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docus Reali
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	Internat TV Sh Romant Shows,

In [7]:
```
df.shape
```

Out[7]: (8807, 12)

In [8]:
```
df.ndim
```

Out[8]: 2

In [9]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   str   
 1   type          8807 non-null   str   
 2   title         8807 non-null   str   
 3   director      6173 non-null   str   
 4   cast          7982 non-null   str   
 5   country       7976 non-null   str   
 6   date_added    8797 non-null   str   
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   str   
 9   duration      8804 non-null   str   
 10  listed_in     8807 non-null   str   
 11  description   8807 non-null   str   
dtypes: int64(1), str(11)
memory usage: 3.9 MB
```

Observation: The data has 8807 rows and 12 columns. Only release_year is a number. All other columns are text, including date_added and duration. Action: I will change date_added to a real date and split duration into a number and a unit.

In [10]:
```
df.isna().sum()
```

Out[10]:
```
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
```

Observation: director (2634), cast (825), country (831), date_added (10), rating (4) and duration (3) have missing values. Action: I will fill director, cast and country with "Unknown", and fix the rating and duration rows.

In [11]:
```
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
```

In [28]:
```
df.isna().sum()
```

Out[28]:
```
show_id          0
type             0
title            0
director         0
cast             0
country          0
date_added      10
release_year     0
rating           0
duration         0
listed_in        0
description      0
dtype: int64
```

In [12]:
```
df[df['duration'].isna()]
```

Out[12]:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	
5541	s5542	Movie	Louis C.K. 2017	Louis C.K.	Louis C.K.	United States	April 4, 2017	2017	74 min	NaN	Movies	or
5794	s5795	Movie	Louis C.K.: Hilarious	Louis C.K.	Louis C.K.	United States	September 16, 2016	2010	84 min	NaN	Movies	
5813	s5814	Movie	Louis C.K.: Live at the Comedy Store	Louis C.K.	Louis C.K.	United States	August 15, 2016	2015	66 min	NaN	Movies	Th hil

In [13]:
```
#Move the value from rating to duration.
#Make rating empty for these rows. Later it will become “Not Rated”.


mask = df['rating'].str.contains('min', na=False)
df.loc[mask, 'duration'] = df.loc[mask, 'rating']
df.loc[mask,'rating'] = np.nan
```

In [14]:
```
df.loc[mask,['title', 'rating','duration']]
```

Out[14]:

	title	rating	duration
5541	Louis C.K. 2017	NaN	74 min
5794	Louis C.K.: Hilarious	NaN	84 min
5813	Louis C.K.: Live at the Comedy Store	NaN	66 min

In [15]:
```
#Fill missing rating values

# Now 7 ratings are empty: 4 were already missing + 3 we emptied in Step 8
# fillna('Not Rated') writes the words "Not Rated" in every empty cell
# Then we save the result back into the same rating column

df['rating'] = df['rating'].fillna('Not rated')
```

In [29]:
```
# Check: count the missing values in every column again

df.isna().sum()
```

Out[29]:
```
show_id          0
type             0
title            0
director         0
cast             0
country          0
date_added      10
release_year     0
rating           0
duration         0
listed_in        0
description      0
dtype: int64
```

In [16]:
```
# to look how date added why is str

df['date_added'].head(10)
```

Out[16]:
```
0    September 25, 2021
1    September 24, 2021
2    September 24, 2021
3    September 24, 2021
4    September 24, 2021
5    September 24, 2021
6    September 24, 2021
7    September 24, 2021
8    September 24, 2021
9    September 24, 2021
Name: date_added, dtype: str
```

In [17]:
```
# Count how many dates start with an extra space
# startswith(' ') is True when the text begins with a space
# sum() counts all the True values
df['date_added'].str.startswith(' ').sum()
```

Out[17]:
```
np.int64(88)
```

In [19]:
```
#Change date_added from text to a real date
# .str.strip() removes extra spaces at the start or end (we found 88 dates with a space)
# pd.to_datetime() changes text like "September 25, 2021" into a real date
# format='mixed' means: read each date on its own, in case they are written a little differently

df['date_added'] = pd.to_datetime(df['date_added'].str.strip(), format='mixed')
```

In [20]:
```
# Make two new columns, year and month, from date_added

# .dt lets us take parts out of a real date
# .dt.year gives the year (like 2021)
# .dt.month gives the month as a number (like 9 for September)

df['year_added'] = df['date_added'].dt.year
df['month_added'] = df['date_added'].dt.month


# Check: Look at the old date and the two new columns together
df[['date_added', 'year_added', 'month_added']].head()
```

Out[20]:

	date_added	year_added	month_added
0	2021-09-25	2021.0	9.0
1	2021-09-24	2021.0	9.0
2	2021-09-24	2021.0	9.0
3	2021-09-24	2021.0	9.0
4	2021-09-24	2021.0	9.0

In [38]:
```
# Look at 10 random values of the duration column

df['duration'].sample(10,random_state=1)
```

Out[38]:
```
2894     1 Season
2161       72 min
6363       74 min
2575     1 Season
2698    2 Seasons
7223       96 min
5759     1 Season
863        88 min
8690    2 Seasons
3625      102 min
Name: duration, dtype: str
```

Observation: In the data set, after data cleaning, I found that the duration column has 2 different kinds of values. One is for movies, where the duration is given directly, like 96 min or 102 min. The other is for TV shows, where it is given in seasons. So I can't find an average if the values are like that.

Action: So I decided to split it into two columns: duration_num and duration_unit.

In [21]:
```
# Split duration into a number and a unit

# split(' ') cuts the text at the space, like "96 min" -> "96" and "min"
# expand=True puts the two parts in two separate columns

df[['duration_num', 'duration_unit']] = df['duration'].str.split(' ',expand =True)

#The number is still text, so change it to a real number

df['duration_num'] = df['duration_num'].astype(int)
```

In [41]:
```
# Check: the old column and the two new columns together

df[['duration', 'duration_num', 'duration_unit']].sample(6, random_state=1)
```

Out[41]:

	duration	duration_num	duration_unit
2894	1 Season	1	Season
2161	72 min	72	min
6363	74 min	74	min
2575	1 Season	1	Season
2698	2 Seasons	2	Seasons
7223	96 min	96	min

In [22]:
```
#Statistical summary of the number columns
# describe() gives count, mean, min, max and the quarters for number columns
# round(1) keeps only 1 digit after the decimal point, so it is easy to read

df[['release_year', 'year_added', 'duration_num']].describe().round(1)
```

Out[22]:

	release_year	year_added	duration_num
count	8807.0	8797.0	8807.0
mean	2014.2	2018.9	69.8
std	8.8	1.6	50.8
min	1925.0	2008.0	1.0
25%	2013.0	2018.0	2.0
50%	2017.0	2019.0	88.0
75%	2019.0	2020.0	106.0
max	2021.0	2021.0	312.0

Observation: In release_year, Netflix has mostly recent data. The oldest title is from 1925, the newest is from 2021, and the middle value (median) is 2017. In duration_num, the numbers are mixed. Movies are in minutes and TV shows are in seasons (1, 2, ...), so the mean (69.8) is not meaningful.

Action: I will look at movies and TV shows separately when I analyse duration.

In [23]:
```
df['country'].head(10)
```

Out[23]:
```
0                                        United States
1                                         South Africa
2                                              Unknown
3                                              Unknown
4                                                India
5                                              Unknown
6                                              Unknown
7    United States, Ghana, Burkina Faso, United Kin...
8                                           United Kingdom
9                                         United States
Name: country, dtype: str
```

In [24]:
```
df.columns
```

Out[24]:
```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
       'release_year', 'rating', 'duration', 'listed_in', 'description',
       'year_added', 'month_added', 'duration_num', 'duration_unit'],
      dtype='str')
```

In [25]:
```
#show all columns without hiding the middle ones

pd.set_option('display.max_columns', None)
df.head(3)
```

Out[25]:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	li
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	Unknown	United States	2021-09-25	2020	PG-13	90 min	Docume
1	s2	TV Show	Blood & Water	Unknown	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	TV-MA	2 Seasons	Interr TV Sh Dra My
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	Unknown	2021-09-24	2021	TV-MA	1 Season	Cr Interr TV Sh

In [26]:
```
# Show the full text of each cell, without cutting it with ...
pd.set_option('display.max_colwidth', None)

# Look at row 7, the country cell
df.loc[7, 'country']
```

Out[26]:
```
'United States, Ghana, Burkina Faso, United Kingdom, Germany, Ethiopia'
```

In [27]:
```
# Un-nest country: cut the text at each comma and put each country in its own row
# First make a small separate table with only the columns we need

country_df = df[['show_id', 'type', 'country']].copy()
```

In [28]:
```
# split(',') cuts the text at each comma and makes a list
# "United States, Ghana" becomes ["United States", " Ghana"]
country_df['country'] = country_df['country'].str.split(',')
```

In [29]:
```
# Check: look at row 7, it should be a list now
country_df.loc[7, 'country']
```

Out[29]:
```
['United States',
 ' Ghana',
 ' Burkina Faso',
 ' United Kingdom',
 ' Germany',
 ' Ethiopia']
```

In [30]:
```
#Put each country in its own row
#Make one row for each country in the list
# explode() takes each item of a list and puts it in its own row
country_df = country_df.explode('country')
```

In [31]:
```
# strip() removes the extra spaces at the start or end of each name
country_df['country'] = country_df['country'].str.strip()
```

In [53]:
```
# Check: how many rows now?
len(country_df)
```

Out[53]: 10850

In [32]:
```
#Look at the result
#Look at the first rows of the new table
# The same title (same show_id) can now appear in more than one row
country_df.head()
```

Out[32]:

	show_id	type	country
0	s1	Movie	United States
1	s2	TV Show	South Africa
2	s3	TV Show	Unknown
3	s4	TV Show	Unknown
4	s5	TV Show	India

In [33]:
```
# Show only the rows of one title (show_id s8)
# == means "is equal to"
country_df[country_df['show_id'] == 's8']
```

Out[33]:

	show_id	type	country
7	s8	Movie	United States
7	s8	Movie	Ghana
7	s8	Movie	Burkina Faso
7	s8	Movie	United Kingdom
7	s8	Movie	Germany
7	s8	Movie	Ethiopia

In [34]:
```
#Count how many titles each country has
# != means "is not equal to", so we remove "Unknown" (it is not a real country)
# value_counts() counts how many times each country appears
# head(10) shows the 10 biggest
country_df[country_df['country'] != 'Unknown']['country'].value_counts().head(10)
```

Out[34]:
```
country
United States     3690
India             1046
United Kingdom     806
Canada             445
France             393
Japan              318
Spain              232
South Korea        231
Germany            226
Mexico             169
Name: count, dtype: int64
```

Observation: In the country column, some titles have many countries written in one cell, like "United States, Ghana, Burkina Faso". So Python counts the whole cell as one name, and the country counts would be wrong. I split the country text at the comma and made one row for each country (un-nesting). Now a title is counted once for each of its countries. For example, one movie (s8) was made in 6 countries, so it appears in 6 rows. The rows grew from 8807 to 10850. The top countries are United States (3690), India (1046) and United Kingdom (806). Action: I will do the same un-nesting for the genre, director and cast columns, because they also have many values in one cell.

In [35]:
```
#Un-nest the genre column (listed_in)
# Make a small table with only the columns we need
genre_df = df[['show_id', 'type', 'listed_in']].copy()
```

In [36]:
```
# Cut the text at each comma, so it becomes a list
genre_df['listed_in'] = genre_df['listed_in'].str.split(', ')
```

In [37]:
```
# Put each genre in its own row
genre_df = genre_df.explode('listed_in')
```

In [38]:
```
# Count the rows
len(genre_df)
```

Out[38]:
```
19323
```

In [39]:
```
# Count how many titles each genre has
genre_df['listed_in'].value_counts().head(10)
```

Out[39]:
```
listed_in
International Movies        2752
Dramas                      2427
Comedies                    1674
International TV Shows      1351
Documentaries                869
Action & Adventure           859
TV Dramas                    763
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616
Name: count, dtype: int64
```

In the genre column, many genres were written in one cell. I cut the text at the comma and put each genre in its own row. Now there are 19323 rows. The top genres are International Movies, Dramas and Comedies. Action: I will do the same for director and cast columns.

In [40]:
```
#Un-nest director
# Take only the 3 columns we need
director_df = df[['show_id', 'type', 'director']].copy()

# Cut the director text at each comma and space
director_df['director'] = director_df['director'].str.split(', ')

# Put each director in its own row

director_df = director_df.explode('director')
```

In [41]:
```
# Count the rows
len(director_df)
```

Out[41]:
```
9612
```

In [42]:
```
# Top 10 directors, without "Unknown"
# != means "is not equal to"
director_df[director_df['director'] != 'Unknown']['director'].value_counts().head(10)
```

Out[42]:
```
director
Rajiv Chilaka          22
Jan Suter              21
Raúl Campos            19
Suhas Kadav            16
Marcus Raboy           16
Jay Karas              15
Cathy Garcia-Molina    13
Youssef Chahine        12
Martin Scorsese        12
Jay Chapman            12
Name: count, dtype: int64
```

In [43]:
```
#Un-nest cast
# Take only the 3 columns we need
cast_df = df[['show_id', 'type', 'cast']].copy()
```

In [44]:
```
# Cut the cast text at each comma and space
cast_df['cast'] = cast_df['cast'].str.split(', ')
```

In [45]:
```
# Put each actor in its own row
cast_df = cast_df.explode('cast')
```

In [46]:
```
# Count the rows
len(cast_df)
```

Out[46]:
```
64951
```

In [47]:
```
# Top 10 actors, without "Unknown"
cast_df[cast_df['cast']!= 'Unknown']['cast'].value_counts().head(10)
```

Out[47]:
```
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Naseeruddin Shah     32
Takahiro Sakurai     32
Rupa Bhimani         31
Akshay Kumar         30
Om Puri              30
Yuki Kaji            29
Amitabh Bachchan     28
Name: count, dtype: int64
```

In [48]:
```
# Count how many different values each column has
df.nunique()
```

Out[48]:
```
show_id         8807
type               2
title           8807
director        4529
cast            7693
country          749
date_added      1714
release_year      74
rating            15
duration         220
listed_in        514
description     8775
year_added        14
month_added       12
duration_num     210
duration_unit      3
dtype: int64
```

Observation: show_id has 8807 different values, so every title appears only once. type has only 2 values (Movie and TV Show), and rating has 15 values. country has 749 different values, because many titles have more than one country written in one cell.

In [49]:
```
# Count how many Movies and how many TV Shows
df['type'].value_counts()
```

Out[49]:
```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```

In [50]:
```
# Show the share of Movies and TV Shows as a percent
# normalize=True gives the share (0 to 1)
# * 100 changes it to percent
# round(1) keeps 1 digit after the decimal point


(df['type'].value_counts(normalize=True)*100).round(1)
```

Out[50]:
```
type
Movie      69.6
TV Show    30.4
Name: proportion, dtype: float64
```

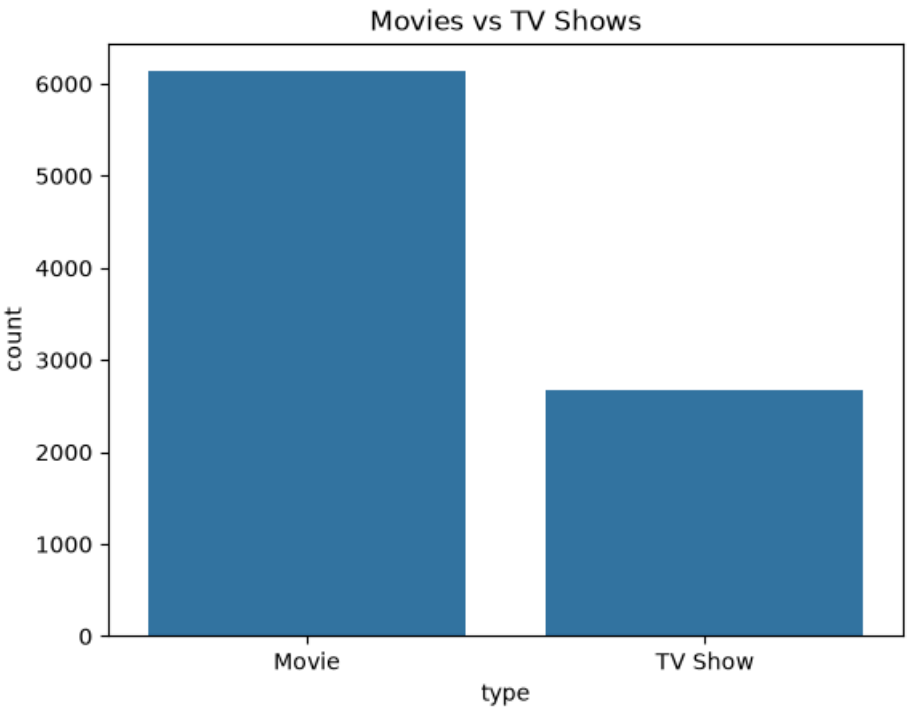
Observation: In the data, movies are 70% (6131 titles) and TV shows are 30% (2676 titles). So Netflix has more movies than TV shows.

Meaning: Movies are the bigger part of Netflix's content.But this does not say that people watch more movies. It only says what Netflix has in its library. We do not have viewing data here.

In [51]:
```
# Draw one bar for each value in the type column
sns.countplot(data=df, x= 'type')
```

```
# Write a title on top of the chart
plt.title('Movies vs TV Shows')


# Show the chart
plt.show()
```



Observation: I see that the Movie bar is much taller than the TV Show bar. Movies are more than twice as many as TV shows.

In [52]:
```
# Draw a histogram of the release year
# bins=12 means: cut the years into 12 groups

ax = sns.histplot(data=df, x='release_year',bins=12)


# Write the count on top of each bar

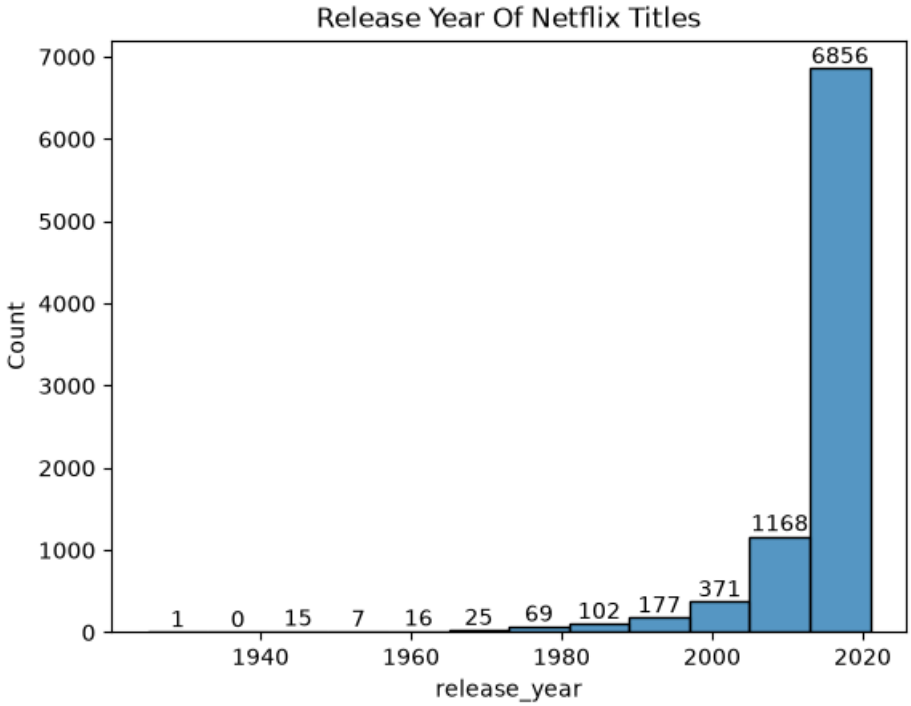
ax.bar_label(ax.containers[0])


# Write a title on top of the chart

plt.title('Release Year Of Netflix Titles')

# Show the chart

plt.show()
```



In [53]:
```
# Percent of titles released in 2013 or later

# Part: titles released in 2013 or later

part = (df['release_year'] >= 2013).sum()

# Total: all titles
total = len(df)

# Percent = part divided by total, times 100
# round(1) keeps 1 digit after the decimal point

round(part / total*100, 1)
```

Out[53]: np.float64(77.8)

Observation: I see that most titles were released between 2013 and 2021. About 6856 titles are in this group. About 78% of all titles (6856 out of 8807) were released between 2013 and 2021.

Meaning: This means Netflix has mostly recent content. Very few old titles are on Netflix.

In [54]:
```
# Keep only the rows where type is Movie
# == means "is equal to"

movies = df[df['type'] == 'Movie']

# Check: how many movies?
len(movies)
```

Out[54]: 6131

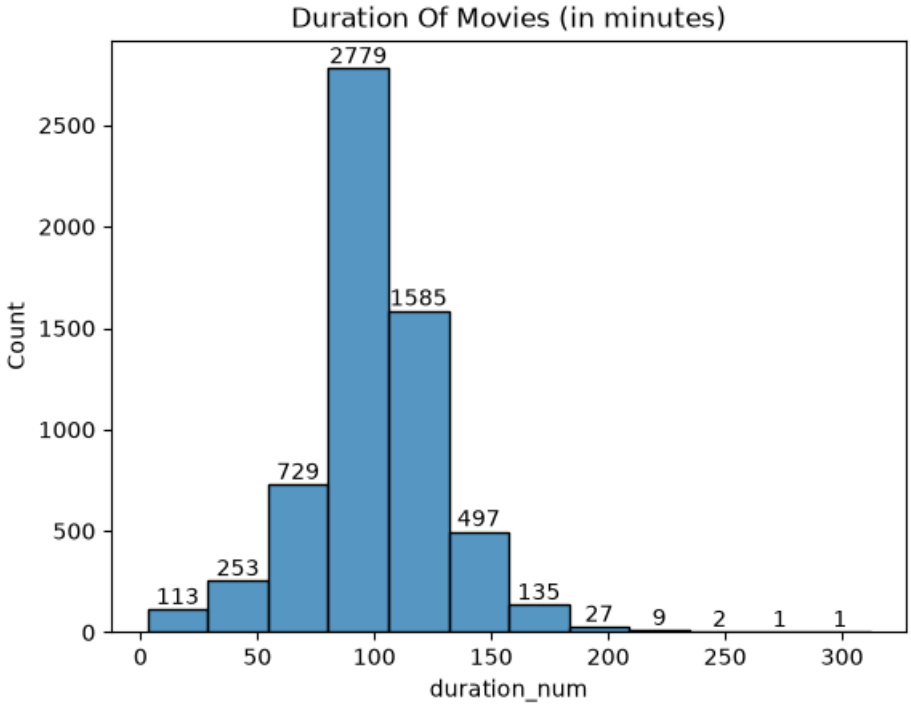
In [122…
```
# Draw a histogram of the movie duration
# bins=12 means: cut the minutes into 12 groups
# ax = gives the chart a name, so we can add numbers to it

ax = sns.histplot(data=movies, x='duration_num', bins=12)

# Write the count on top of each bar
ax.bar_label(ax.containers[0])

# Write a title on top of the chart
plt.title('Duration Of Movies (in minutes)')

#show the chart
plt.show()
```



Observation: I see that the tallest bar has 2779 movies, around 80 to 106 minutes. The next bar has 1585 movies, around 106 to 132 minutes. The chart has a long, thin tail on the right, because a few movies are very long (up to about 300 minutes).

Meaning: This means Most Netflix movies are about one and a half hours long, and a few are very long, most movies are roughly 1 to 2 hours long. The three middle bars (729 + 2779 + 1585) make 5093 movies, which is about 83% of all movies. The median movie is 98 minutes.

In [55]:
```
# Keep only the rows where type is TV Show
# == means "is equal to"

tv_shows = df[df['type'] == 'TV Show']

# Check: how many TV shows?
len(tv_shows)
```

Out[55]: 2676

In [128…
```
# Draw one bar for each number of seasons
# ax = gives the chart a name, so we can add numbers to it

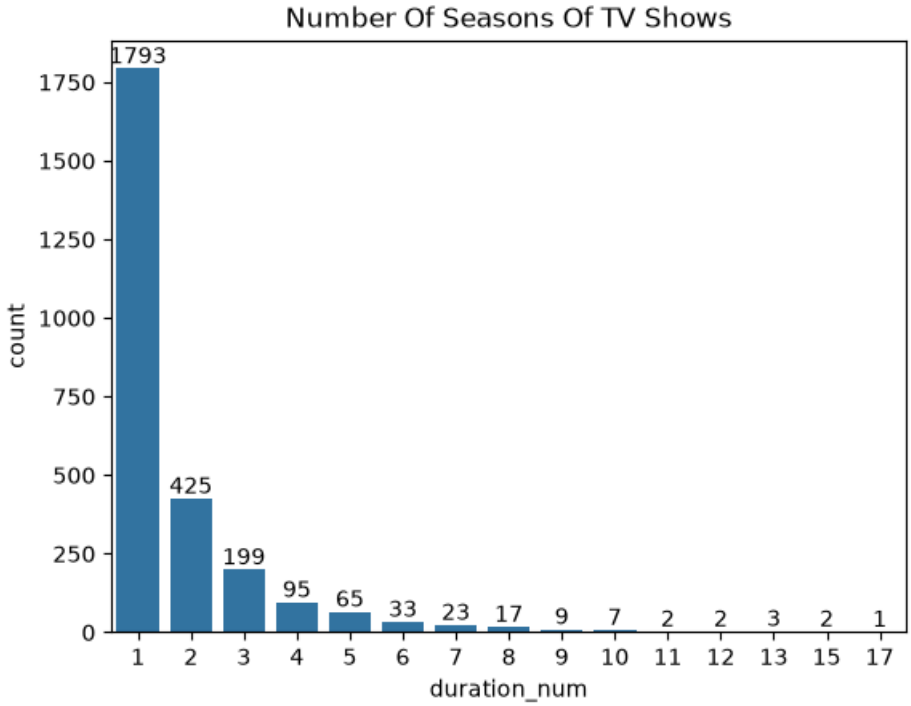
ax = sns.countplot(data=tv_shows, x='duration_num')

# Write the count on top of each bar

ax.bar_label(ax.containers[0])

# Write a title on top of the chart
plt.title('Number Of Seasons Of TV Shows')

# Show the chart
plt.show()
```



In [56]:
```
# Percent of TV shows with 1 season
# (tv_shows['duration_num'] == 1) is True for 1-season shows, and mean() gives the share
# * 100 changes it to percent, and round(1) keeps 1 digit after the decimal point
round((tv_shows['duration_num'] == 1).mean() * 100, 1)
```

Out[56]: np.float64(67.0)

Observation: I see that 1793 TV shows have just 1 season. This is 67%, about 2 out of 3 TV shows. Reading the chart to the right, the number of shows gets smaller. Only 1 TV show has 17 seasons. Most shows are in 1, 2 or 3 seasons. Shows with many seasons are very few.

Meaning: This means most TV shows on Netflix are short, with only a few seasons.

Note: the data shows how many seasons a show has now, so we cannot say why a show is short.

In [57]:
```
# Countplot of the rating (We want to know who Netflix makes content for: adults, teenagers or kids.)

# Draw one bar for each rating
# y='rating' puts the names on the left side, so they are easy to read
# order=... sorts the bars from the biggest to the smallest
# ax = gives the chart a name, so we can add numbers to it

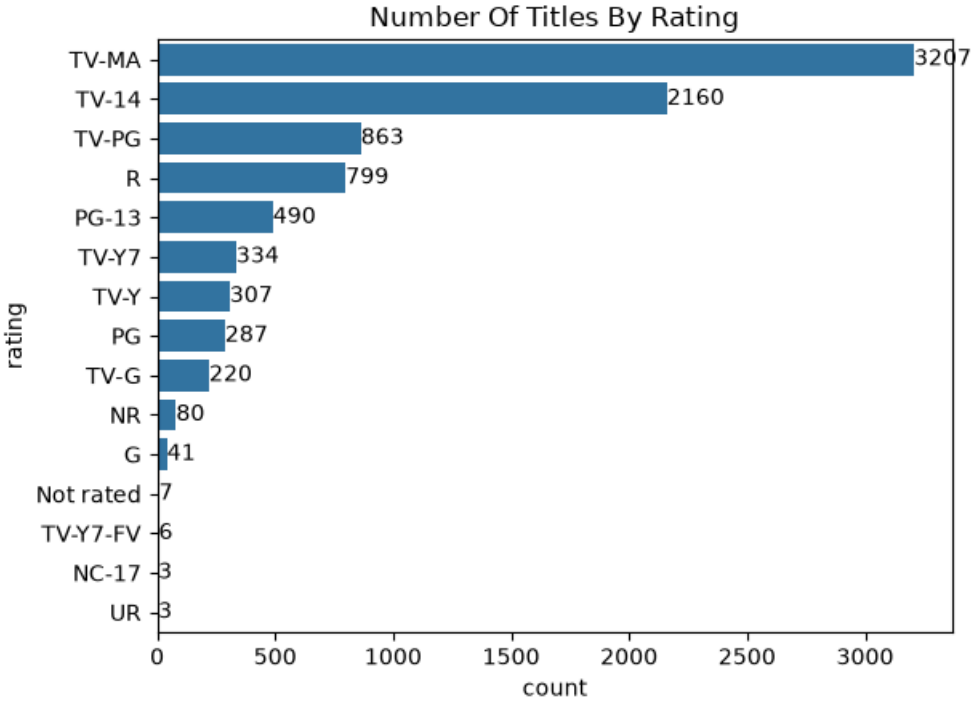
ax= sns.countplot(data=df, y='rating', order =df['rating'].value_counts().index)

# Write the count at the end of each bar

ax.bar_label(ax.containers[0])

# Write a title on top of the chart
plt.title('Number Of Titles By Rating')

# Show the chart
plt.show()
```



Observation: I see that TV-MA (adults) is the highest with 3207 titles, followed by TV-14 with 2160 and TV-PG with 863. The lowest are NC-17 and UR, with 3 titles each.

Meaning: TV-MA and TV-14 together are about 61% of all titles. This means Netflix content is mostly for adults and teenagers.

note: The rating column has both NR (80 titles) and Not rated (7 titles). Both mean that no rating was given, so they can be treated as one group.

In [58]:
```
#the top 10 countries chart
# Count the top 10 countries, without "Unknown"
# != means "is not equal to"

top_countries = country_df[country_df['country']!='Unknown']['country'].value_counts().head(10)

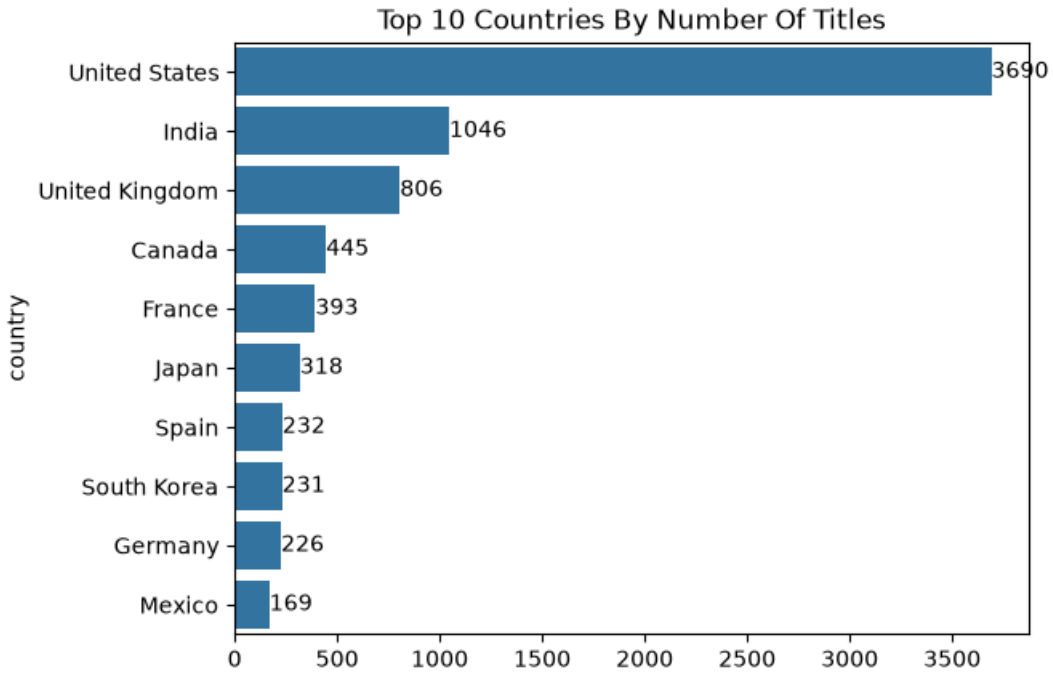

# Draw one bar for each of the top 10 countries
# ax = gives the chart a name, so we can add numbers to it

ax = sns.barplot(x=top_countries.values, y=top_countries.index)

# Write the count at the end of each bar
ax.bar_label(ax.containers[0])

# Write a title on top of the chart
plt.title('Top 10 Countries By Number Of Titles')


# Show the chart
plt.show()
```



Observation: I see that the United States has the most titles (3690), followed by India (1046) and the United Kingdom (806). The other 7 countries in the top 10 have fewer titles, and Mexico has the lowest with 169.

Meaning: This means the United States has the biggest share of Netflix titles, about 4 out of 10 (42%), and it is far ahead of all other countries. India is second, so Indian content is a big part of Netflix. Content also comes from many other countries.

Note: A title made in many countries is counted once for each country, so these counts can add up to more than the number of titles. This chart shows where titles were made, not where people watch them. So we can say "Netflix has a lot of content from the US and India", but we cannot say "most viewers are in the US".

In [59]:
```
#the top 10 genres chart
# Count the top 10 genres
top_genres = genre_df['listed_in'].value_counts().head(10)

# Draw one bar for each of the top 10 genres
# ax = gives the chart a name, so we can add numbers to it
ax = sns.barplot(x=top_genres.values, y=top_genres.index)


# Write the count at the end of each bar

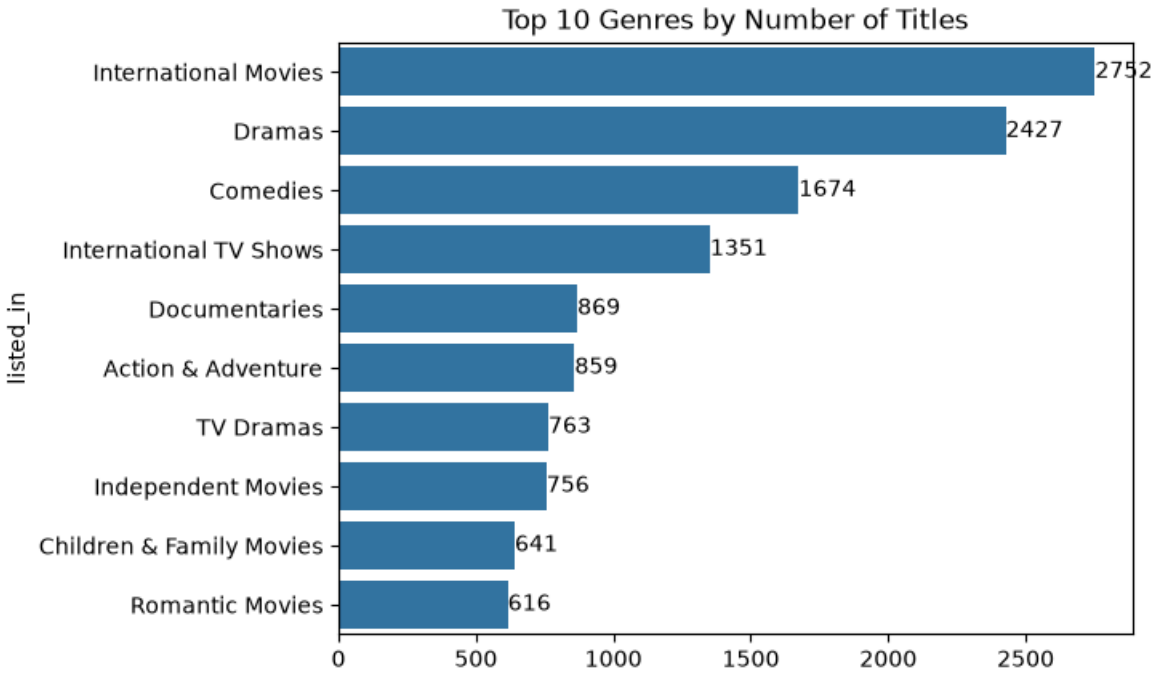
ax.bar_label(ax.containers[0])

# Write a title on top of the chart

plt.title('Top 10 Genres by Number of Titles')


# Show the chart

plt.show()
```



Observation: I see that International Movies has the most titles (2752), followed by Dramas (2427) and Comedies (1674). The other 7 genres follow, and Romantic Movies has the lowest in the top 10, with 616 titles.

Meaning: This means Netflix has the most titles in International Movies, Dramas and Comedies. 8 of the top 10 genres are movie genres and only 2 are TV genres, which fits with movies being about 70% of all titles.

Note: A title can have more than one genre, so it is counted once for each genre.

In [60]:
```
#Titles added each year, Movies vs TV Shows
# Keep only the titles added in 2014 or later
# >= means "is greater than or equal to"

recent = df[df['year_added'] >= 2014].copy()

#Change the year from 2021.0 to 2021 (a whole number)

recent['year_added'] = recent['year_added'].astype(int)

# Draw two bars for each year: one for Movies and one for TV Shows
# hue='type' gives each type its own colour

ax = sns.countplot(data=recent, x='year_added',hue='type')

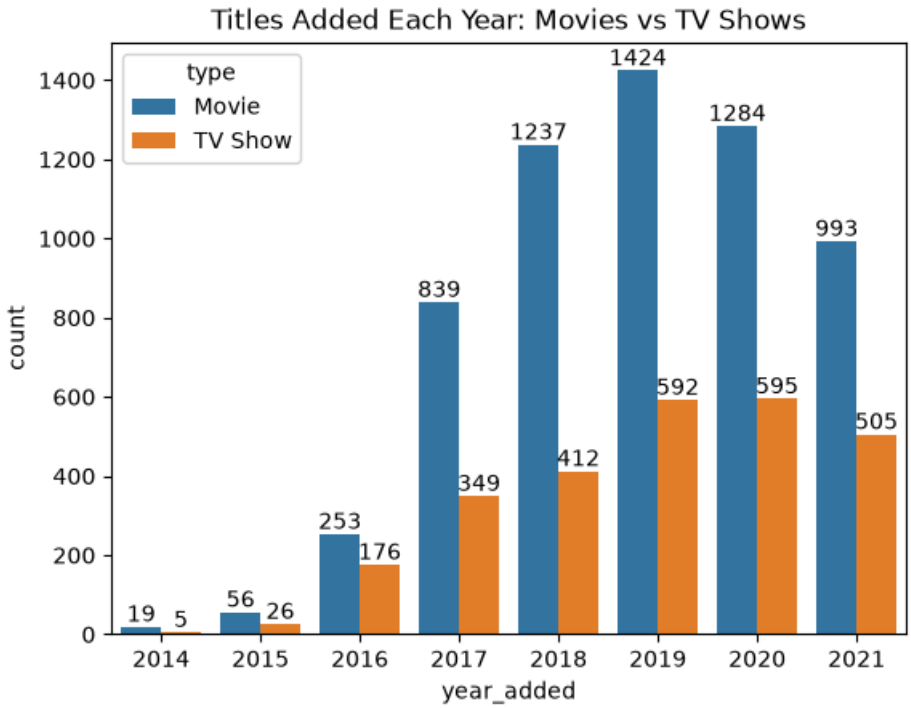
# Write the count on top of each bar
# There are 2 sets of bars (Movie and TV Show), so we go through both

for bars in ax.containers:
    ax.bar_label(bars)

# Write a title on top of the chart

plt.title('Titles Added Each Year: Movies vs TV Shows')


# Show the chart
plt.show()
```



Observation: I see that the most movies were added in 2019, with 1424 titles. After 2019, the number of movies went down: 1284 in 2020 and 993 in 2021. TV shows stayed about the same in 2019 and 2020 (592 and 595), and went down to 505 in 2021.

Meaning: This means Netflix added the most titles in 2019, and fewer movies after that. The share of TV shows in new titles went up from 25% in 2018 to 33.7% in 2021. So Netflix is slowly adding more TV shows compared to movies.

Note: The data ends on 25 September 2021, so 2021 is not a full year. The lower numbers for 2021 are partly because of this.

In [61]:
```
#Titles added each month, Movies vs TV Shows

# Keep only the rows that have a month (10 rows have no date)
# dropna(subset=[...]) removes the rows where month_added is empty

months = df.dropna(subset=['month_added']).copy()

# Change the month from 9.0 to 9 (a whole number)
months['month_added'] = months['month_added'].astype(int)

# Count how many Movies and TV Shows were added in each month
# groupby puts the rows into groups of (month, type)
# size() counts the rows in each group
# unstack() makes one column for Movie and one for TV Show

monthly = months.groupby(['month_added', 'type']).size().unstack()


# Look at the table first
monthly
```

Out[61]:

type	Movie	TV Show
month_added		
1	546	192
2	382	181
3	529	213
4	550	214
5	439	193
6	492	236
7	565	262
8	519	236
9	519	251
10	545	215
11	498	207
12	547	266

In [146…

```
# draw the chart

# Draw one line for Movies and one for TV Shows
# marker='o' puts a dot on each month
monthly.plot(marker='o')

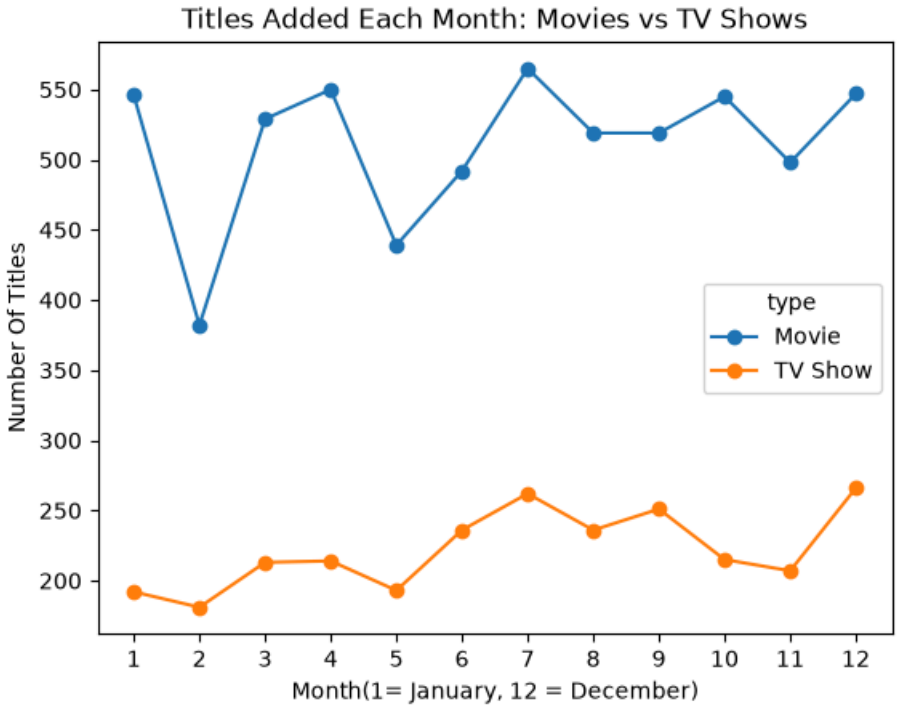
# Show every month number (1 to 12) on the bottom line

plt.xticks(range(1,13))

# Write a title on top of the chart
plt.title('Titles Added Each Month: Movies vs TV Shows')

# Write names on the two sides of the chart
plt.xlabel('Month(1= January, 12 = December)')
plt.ylabel('Number Of Titles')

#show  the charts
plt.show()
```



Observation: I see that movies are added the most in July (565 titles), followed by April (550) and December (547). TV shows are added the most in December (266 titles), followed by July (262) and September (251). February is the lowest month for both: 382 movies and 181 TV shows.

Meaning: This means Netflix adds new titles through the whole year, with a little more in July and December. For both movies and TV shows, July and December are the busiest months, and February is the quietest.

Note: The difference between most months is small. The chart starts above zero on the left side, so the dips look bigger than they are. February is also a shorter month, so a lower number is expected.

In [62]:
```
#Movies vs TV Shows in the top 10 countries

# Take the names of the top 10 countries (without Unknown)
top10 = country_df[country_df['country']!='Unknown']['country'].value_counts().head(10).index

# Keep only the rows of these 10 countries
top_df = country_df[country_df['country'].isin(top10)]

# Count how many Movies and TV Shows each country has
# groupby puts the rows into groups of (country, type), and size() counts them
# unstack() makes one column for Movie and one for TV Show
# .loc[top10] keeps the countries in the same order as the top 10

table = top_df.groupby(['country', 'type']).size().unstack().loc[top10]

# Add a column: the percent of TV Shows in each country
table['TV%'] = (table['TV Show'] / (table['Movie'] + table['TV Show']) * 100).round(1)

# Look at the table
table
```

Out[62]:

type country	Movie	TV Show	TV%
United States	2752	938	25.4
India	962	84	8.0
United Kingdom	534	272	33.7
Canada	319	126	28.3
France	303	90	22.9
Japan	119	199	62.6
Spain	171	61	26.3
South Korea	61	170	73.6
Germany	182	44	19.5
Mexico	111	58	34.3

Observation: I see that Japan (62.6%) and South Korea (73.6%) have more TV shows than movies. India has mostly movies: 962 movies and only 84 TV shows (8% TV shows). The other countries (United States, United Kingdom, Canada, France, Spain, Germany and Mexico) have more movies than TV shows. Their TV show share is between about 20% and 34%.

Meaning: This means different countries make different types of content. India is strong in movies, and Japan and South Korea are strong in TV shows.

Note: This table shows what each country makes, not what people there watch. A title made in many countries is counted once for each country

In [63]:
```
# Draw two bars for each country: one for Movies and one for TV Shows
# order=top10 keeps the countries from the biggest to the smallest
# hue='type' gives each type its own colour

ax =sns.countplot(data=top_df, y='country', hue='type', order=top10)
```

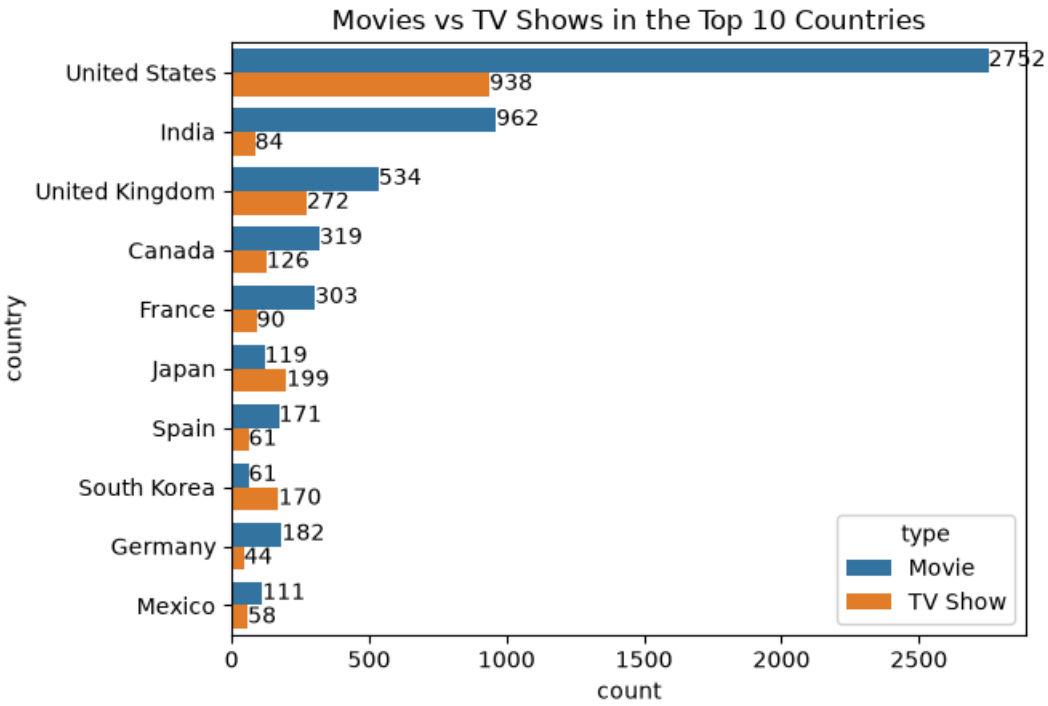
```
# Write the count at the end of each bar (for both colours)

for bars in ax.containers:
    ax.bar_label(bars)


# Write a title on top of the chart

plt.title('Movies vs TV Shows in the Top 10 Countries')

# Show the chart
plt.show()
```



In [64]:
```
# Join the two tables using show_id (the unique number of each title)
# merge() puts the country and the genre of the same title in one row
country_genre = country_df.merge(genre_df[['show_id', 'listed_in']],on= 'show_id' , how='inner')

# Check: how many rows now?
len(country_genre)
```

Out[64]:
```
23764
```

In [65]:
```
#Then the genres of 3 countries
# Top 5 genres of the United States
# [country_genre['country'] == 'United States'] keeps only the rows of that country
country_genre[country_genre['country'] == 'United States']['listed_in'].value_counts().head(5)
```

Out[65]:
```
listed_in
Dramas                  835
Comedies                680
Documentaries           512
Action & Adventure      404
Independent Movies      390
Name: count, dtype: int64
```

In [66]:
```
# Top 5 genres of India
country_genre[country_genre['country'] == 'India']['listed_in'].value_counts().head(5)
```

Out[66]:
```
listed_in
International Movies    864
Dramas                  662
Comedies                323
Independent Movies      167
Action & Adventure      137
Name: count, dtype: int64
```

In [67]:
```
# Top 5 genres of South Korea
country_genre[country_genre['country'] == 'South Korea']['listed_in'].value_counts().head(5)
```

Out[67]:
```
listed_in
International TV Shows    152
Korean TV Shows           132
Romantic TV Shows          77
International Movies       44
TV Dramas                  38
Name: count, dtype: int64
```

Observation: I see that in the United States, the top genre is Dramas (835), followed by Comedies (680) and Documentaries (512). In India, the top is International Movies (864), followed by Dramas (662) and Comedies (323). In South Korea, the top is International TV Shows (152), followed by Korean TV Shows (132) and Romantic TV Shows (77).

Meaning: This means Dramas and Comedies are popular genres in the United States and India. South Korea is different: its top genres are TV shows, and Romantic TV Shows is in the top 3.

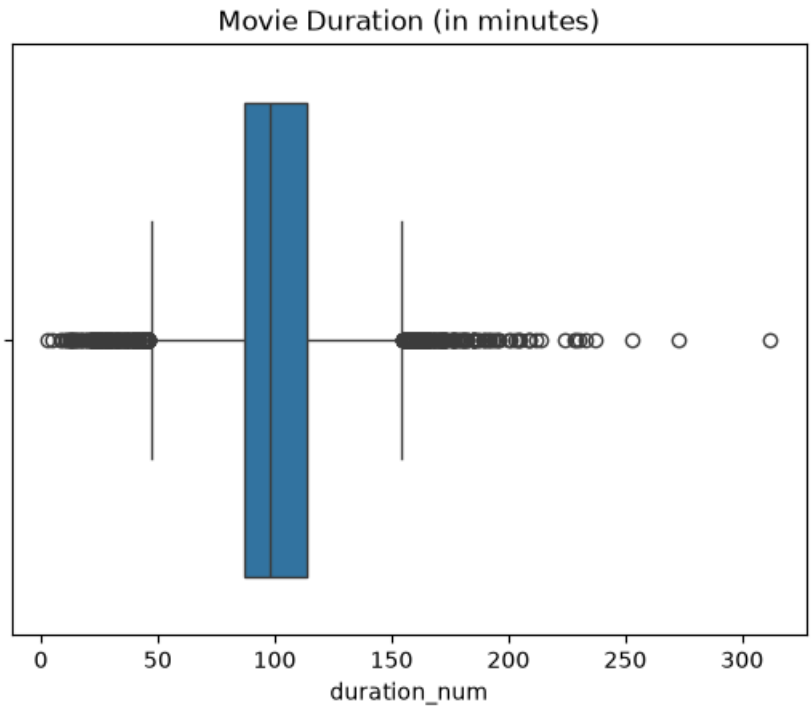
Note: International Movies and International TV Shows are labels for content made outside the US, so they show where a title comes from more than the type of story. This shows what each country makes, not what viewers watch.

In [68]:
```
#check for outliers
# Draw a boxplot of the movie duration
# The box shows where the middle half of the movies are
# The dots outside the lines are outliers (very short or very long movies)

sns.boxplot(data = movies, x ='duration_num')

# Write a title on top of the chart
plt.title('Movie Duration (in minutes)')

# show the chart
plt.show()
```



Observation: I see that the box is between 87 and 114 minutes, and the middle value (median) is 98 minutes. There are outliers on both sides: very short movies (down to 3 minutes) and very long movies (up to 312 minutes).

Meaning: This means the middle half of the movies are between about 1.5 and 2 hours long. The outliers are real values, such as short films and very long movies, not mistakes, so we keep them.

In [69]:
```
# Q1 = the value below which 25% of the movies fall, Q3 = the value below which 75% fall
q1 = movies['duration_num'].quantile(0.25)
q3 = movies['duration_num'].quantile(0.75)
```

```
# IQR = the size of the box (Q3 minus Q1)  (IQR = Interquartile Range.)
iqr = q3 - q1

# A value is an outlier if it is more than 1.5 boxes away from the box
low = q1 -1.5 * iqr
high = q3 + 1.5 * iqr

# Count the movies that are shorter than low, or longer than high
outliers = movies[(movies['duration_num'] < low) |(movies['duration_num'] > high)]
len(outliers)
```

Out[69]: 450

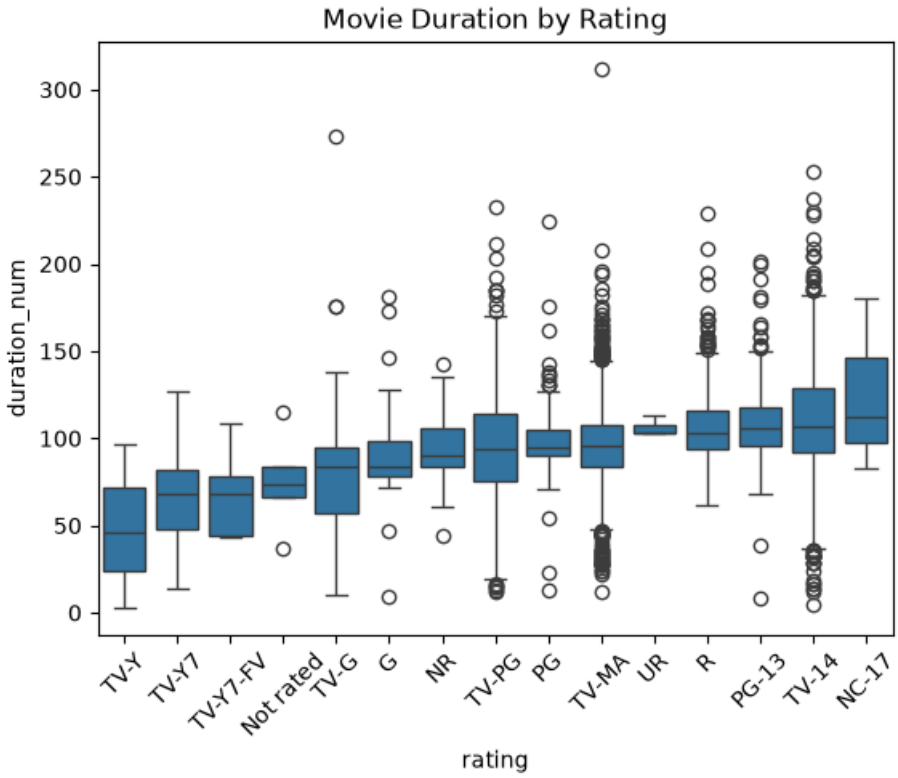
Outliers were found with the IQR rule (the standard 1.5 × IQR rule used by boxplots). They are real movies (very short or very long), so we keep them. 450 movies (7.3%) are outliers

In [70]:
```
# Draw a boxplot of the movie duration for each rating
# order=... puts the ratings from the shortest median movie to the longest
order = movies.groupby('rating')['duration_num'].median().sort_values().index
sns.boxplot(data=movies, x='rating', y='duration_num', order=order)

# Write a title on top of the chart
plt.title('Movie Duration by Rating')

# Turn the names on the bottom a little, so they do not touch
plt.xticks(rotation=45)

# Show the chart
plt.show()
```



Observation: I see that kids' movies are the shortest. The middle value (median) is 46 minutes for TV-Y, 68 for TV-Y7, and 84 for TV-G and G. Movies for teens and adults are longer: TV-MA 96, R 103, PG-13 106 and TV-14 107 minutes. Meaning: This means movies for kids are shorter than movies for teens and adults. Note: Some ratings have very few movies (NC-17 has 3, UR has 3), so their boxes are not reliable.

Business Insights

Insight 1: Movies are about 70% of Netflix titles. There are 6131 movies and 2676 TV shows, so movies are about 70% and TV shows about 30%.

NOTE : This is what Netflix has, not what people watch. The data has no viewing numbers.

Insight 2: TV shows have been growing since 2018. The share of TV shows in new titles went from 25% in 2018 to 34% in 2021, and movie additions went down after 2019.

In [71]:
```
# Count the Movies and TV Shows added in each year (we use the table "recent" from Step 30)
yearly = recent.groupby(['year_added', 'type']).size().unstack()

# Add a column: the percent of TV Shows among all titles added that year
yearly['TV %'] = (yearly['TV Show'] / (yearly['Movie'] + yearly['TV Show']) * 100).round(1)

# Look at the table
yearly
```

Out[71]:

type	Movie	TV Show	TV %
year_added			
2014	19	5	20.8
2015	56	26	31.7
2016	253	176	41.0
2017	839	349	29.4
2018	1237	412	25.0
2019	1424	592	29.4
2020	1284	595	31.7
2021	993	505	33.7

Note: 2021 has data only up to 25 September, so its counts are lower. The share is still fair to compare, because it uses a part of the year for both movies and TV shows

Insight 3: Most TV shows are short. 67% of TV shows (1793 out of 2676) have only 1 season, and about 90% have 3 seasons or fewer.

In [72]:
```
# Percent of TV shows with 3 seasons or fewer
# (tv_shows['duration_num'] <= 3) is True for these shows, and mean() gives the share
# * 100 changes it to percent, and round(1) keeps 1 digit after the decimal point
round((tv_shows['duration_num'] <= 3).mean() * 100, 1)
```

Out[72]:
```
np.float64(90.3)
```

Note: The data shows how many seasons a show has now. We cannot say why: it may be new, or it may have ended early.

Insight 4: Netflix has content from many countries. Titles were made in 122 countries. The United States has the biggest share (3690 titles, about 4 out of 10), followed by India (1046) and the United Kingdom (806).

In [73]:
```
# Keep only real countries (remove "Unknown" and empty names)
real = country_df[(country_df['country'] != 'Unknown') & (country_df['country'] != '')]

# Count how many different countries there are
# nunique() counts each different name only once
real['country'].nunique()
```

Out[73]:
```
122
```

Note: This shows where titles were made, not where people watch them. A title with many countries is counted for each of them.

Insight 5: Countries are different. India makes mostly movies (92%), while South Korea (74%) and Japan (63%) make mostly TV shows.

Note: This shows what each country makes, not what people there watch. A title made in many countries is counted for each of them.

Insight 6: Content is mostly for adults and teens. TV-MA and TV-14 are about 61% of all titles, while kids' ratings are only about 10%.

In [74]:
```
# Share of titles with the ratings TV-MA (adults) or TV-14 (teens)
# isin([...]) is True when the rating is in this list, and mean() gives the share
# * 100 changes it to percent, and round(1) keeps 1 digit after the decimal point

round(df['rating'].isin(['TV-MA','TV-14']).mean() * 100,1)
```

Out[74]:
```
np.float64(60.9)
```

In [75]:
```
# Share of titles with a kids rating
round(df['rating'].isin(['TV-Y', 'TV-Y7', 'TV-Y7-FV', 'TV-G', 'G']).mean() * 100, 1)
```

Out[75]:
```
np.float64(10.3)
```

Note: A rating shows who a title is made for, not who watches it.

Insight 7: International Movies, Dramas and Comedies are the top genres. They have 2752, 2427 and 1674 titles. 8 of the top 10 genres are movie genres and only 2 are TV genres.

Insight 8: A normal movie is about 98 minutes long. The middle half of the movies are between 87 and 114 minutes. Only 7.3% of the movies are very short or very long.

Insight 9: Netflix has mostly recent content. About 78% of the titles (6856 out of 8807) were released between 2013 and 2021.

Insight 10: Netflix adds titles through the whole year. July (827 titles) and December (813) are the busiest months, and February (563) is the quietest. The differences between most months are small.

Recommendations

Recommendation 1: Make more TV shows, starting with 1-season shows. Reason: The share of TV shows in new titles is growing (25% in 2018 to 34% in 2021), and most TV shows (67%) have only 1 season, so Netflix can try a show with 1 season and continue only the ones that do well.

Recommendation 2: Add more TV shows from South Korea and Japan. Reason: South Korea (74%) and Japan (63%) already make mostly TV shows, so these countries are strong in TV. Netflix can add more of their shows.

Recommendation 3: Add more drama movies from India. Reason: India makes mostly movies (92% of its titles), and Dramas is one of its top genres (662 titles, about 63% of India's titles). Netflix can keep building on this strength.

Recommendation 4: Add more content for kids and families. Reason: Kids ratings are only about 10% of all titles, while TV-MA and TV-14 are about 61%. Kids and family content is a small part of the library, so there may be room to grow.

Recommendation 5: Keep making local-language content in many countries. Reason: "International Movies" (2752 titles) and "International TV Shows" (1351 titles) are among the top genres, so content from outside the US is already a big part of Netflix. More local content can help Netflix grow in different countries.

Recommendation 6: Keep International Movies, Dramas and Comedies as the core genres. Reason: They are the top 3 genres, with 2752, 2427 and 1674 titles, and 8 of the top 10 genres are movie genres. Netflix already has a strong base here, so it can keep building on it.

Recommendation 7: Launch big TV shows in July and December, and use quiet months like February for smaller titles. Reason: July (827 titles) and December (813) are the busiest months, and February (563) is the quietest. The gaps are small, so Netflix should test this first.

Recommendation 8: Collect viewing data, such as hours watched in each country. Reason: This data shows what Netflix has, not what people watch. Viewing data can confirm which of these ideas will work.

In [ ]: